CAMPUSGUARD / SENTINEL

Cahier des charges, analyse et architecture

Mini-projet Big Data – Scénario C : sécurité et authentification

Syrine Maaref | IT Business School | 2026

1. Cahier des charges

1.1. Problématique et objectifs

Les applications, serveurs et équipements réseau produisent des journaux de sécurité dispersés. Leur consultation manuelle complique la recherche d'événements suspects et retarde les investigations.

Problématique : comment centraliser ces traces, identifier rapidement des comportements à risque et permettre à un analyste de comprendre et traiter un incident ?

Objectif : concevoir **CampusGuard / Sentinel**, une application web de supervision des logs reposant sur ELK et NoSQL, avec détection progressive des anomalies.

1.2. Objectifs et utilisateurs cibles

Le projet vise trois résultats : **collecter et indexer** des journaux hétérogènes ; **explorer et visualiser** les événements ; **détecter et tracer** les alertes de sécurité. L'accès aux actions dépend du rôle de chaque utilisateur.

Acteur	Responsabilité
Analyste SOC	Surveiller les logs, investiguer et traiter les alertes.
Administrateur	Gérer les comptes, les sources, les règles et les IP bloquées.
Auditeur	Consulter les journaux, l'historique et les rapports.

1.3. Périmètre et priorités

Le **MVP obligatoire** comprend l'importation, la normalisation, la recherche, la visualisation et les métadonnées. Les modules avancés ajoutent les alertes, la réponse temporaire, le temps réel et des expérimentations ML/NLP. Le blocage s'applique uniquement à l'application de démonstration, jamais à un réseau tiers.

1.4. Besoins fonctionnels

Module	Fonctionnalités attendues	Phase
Accès	Authentification et gestion des droits par rôle.	MVP
Ingestion	Import CSV/JSON/TXT, contrôle, aperçu, suivi, parsing et indexation.	MVP
Exploration	Recherche et filtres, détail des logs, export CSV et dashboard.	MVP
Détection	Règles de sécurité, alertes, notifications et gestion d'incidents.	Avancé
Réponse	Blocage temporaire d'IP avec expiration et traçabilité, dans le laboratoire.	Avancé
Temps réel	Collecte continue et affichage des événements en direct.	Avancé
Innovation	Comparaisons expérimentales ML et NLP sur données étiquetées.	Optionnel

1.5. Besoins non fonctionnels

Sécurité	Gestion des rôles, secrets hors du code, audit des actions sensibles.
Fiabilité	Rejets explicites, vérification de santé des services et reprise après erreur.
Ergonomie	Interface responsive et parcours cohérents avec les maquettes.
Performance	Recherche paginée, latence et mémoire mesurées sur la machine cible.
Portabilité	Déploiement Docker Compose documenté et reproductible.

1.6. Données de sécurité

Les données proviennent de **Loghub OpenSSH** (historique réel), d'un **simulateur étiqueté** (authentification, WAF, pare-feu, audit admin) et, éventuellement, de **Cowrie** sur un VPS isolé et autorisé. Sans VPS, le flux temps réel est simulé par rejeu, explicitement signalé comme tel.

Les champs communs normalisés sont : horodatage, service, niveau, message, IP source et utilisateur lorsqu'ils existent. Cette cohérence rend les filtres et la détection comparables entre les sources.

2. Analyse fonctionnelle

Le système relie trois parcours : **ingérer un fichier**, **explorer les événements** et **traiter une alerte**. L'analyste peut examiner les logs associés à un incident, puis enregistrer une décision. L'administrateur configure les règles et les actions sensibles ; l'auditeur consulte les traces sans les modifier.

2.1. Scénarios prioritaires

Cas d'utilisation	**Comportement attendu**
Importer un journal	Vérification du format, prévisualisation des dix premières lignes, indexation ou rejet motivé ; le statut reste consultable.
Rechercher un événement	Filtres (date, gravité, service, IP), résultats paginés, détail et export selon les droits.
Traiter une alerte	Consultation des preuves, acquittement, qualification, résolution ou escalade ; historique conservé.
Bloquer temporairement	Après un seuil configurable, inscription d'une IP dans Redis avec TTL, audit dans MongoDB et déblocage autorisé.

2.2. Règles métier

— Un fichier invalide n'est pas déclaré indexé ; l'erreur de validation ou de parsing est visible.
— Une alerte possède une source, une gravité, un statut et la chronologie des décisions.
— Le scénario de référence est **cinq échecs SSH depuis une même IP en cinq minutes** ; les seuils sont configurables.
— Tout blocage automatique est **temporaire, traçable et limité au laboratoire**.

3. Diagramme de cas d'utilisation UML

Fonctionnalités et rôles dans CampusGuard / Sentinel.

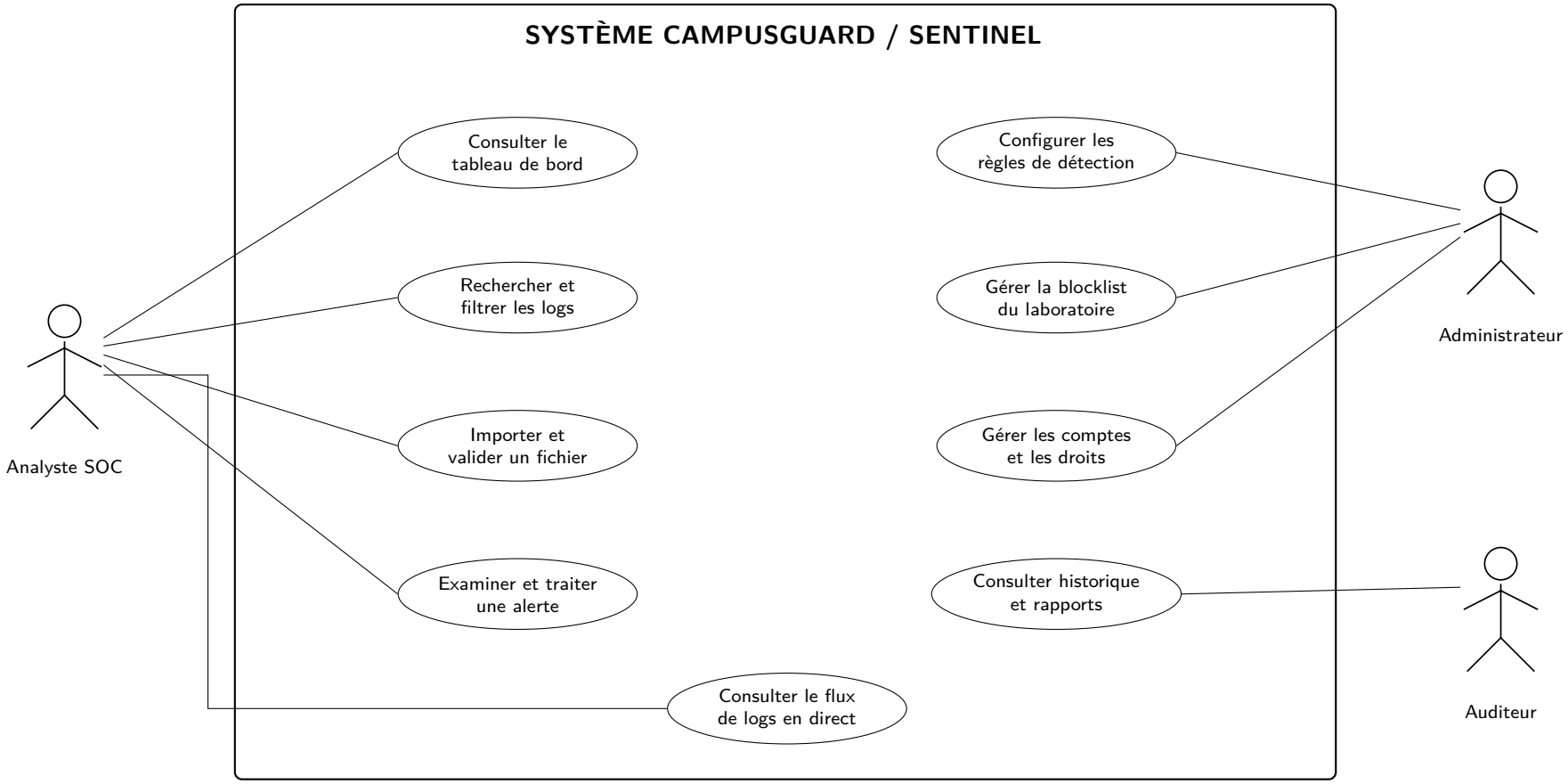


Figure 1 – Cas d'utilisation et acteurs.

4. BPMN – Traitement d'un incident de sécurité

Parcours de traitement d'une alerte, de sa détection à sa clôture.

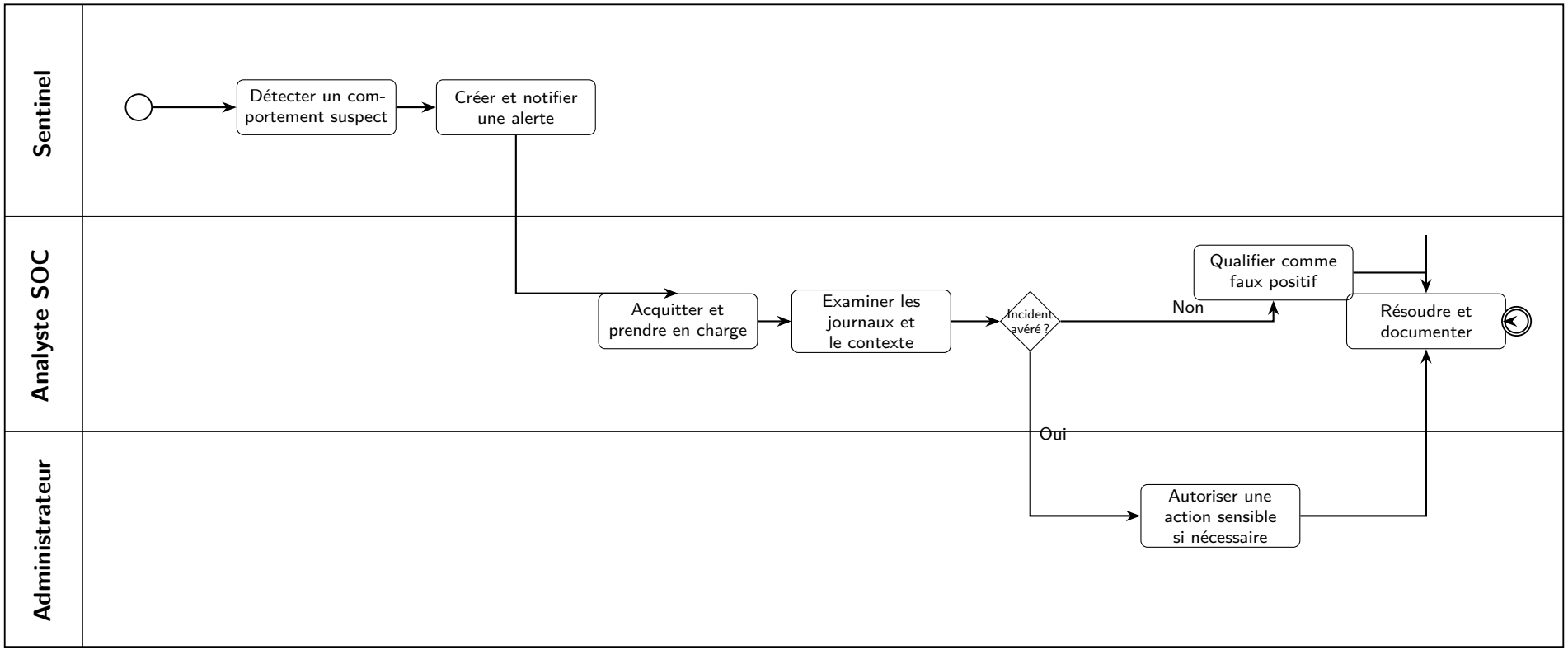


Figure 2 – Traitement d'un incident et responsabilités.

5. Diagramme de séquence UML – Détection d'une force brute SSH

Scénario de référence : cinq authentifications SSH échouées depuis une même IP en cinq minutes (mode STREAM).

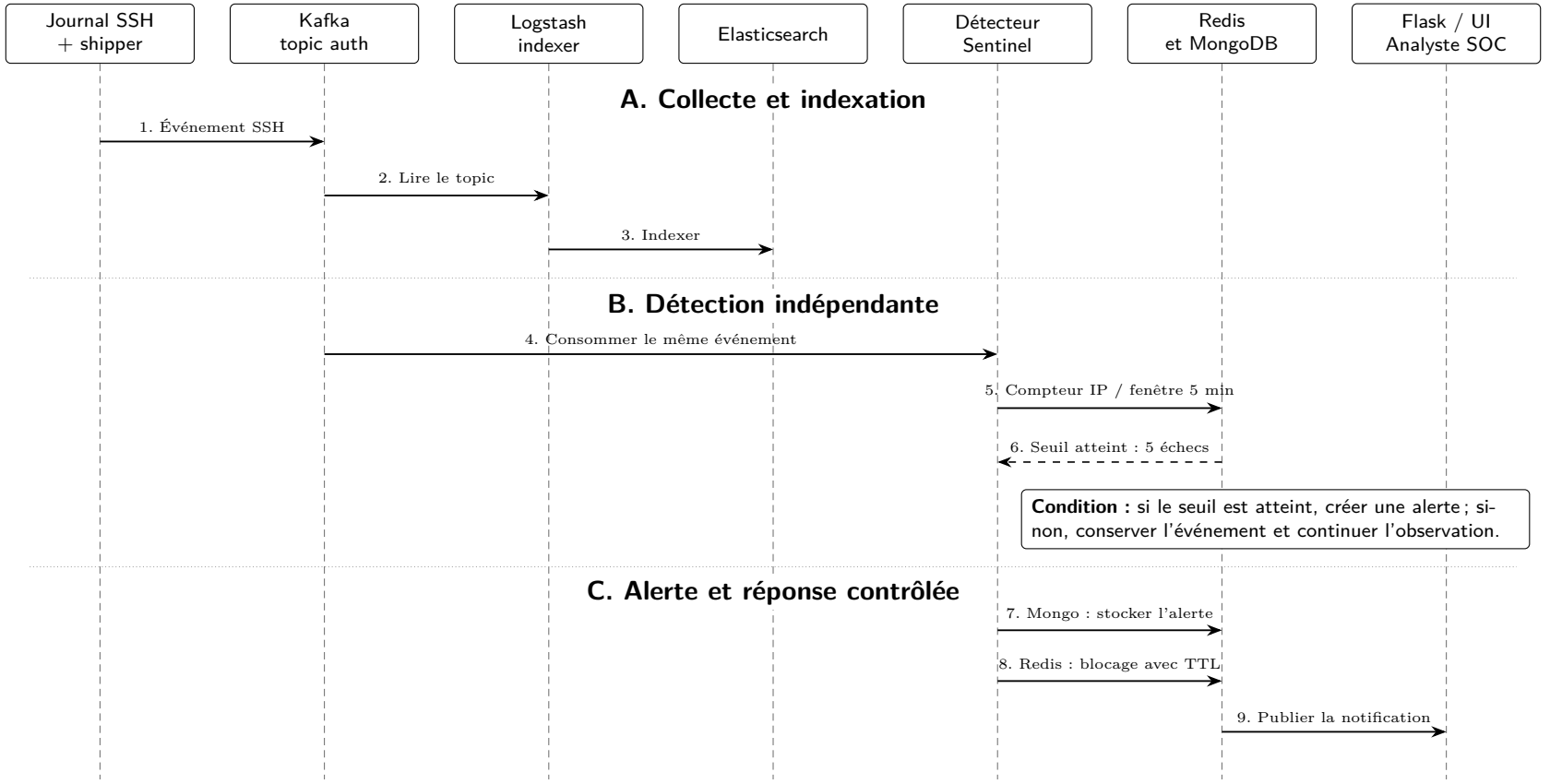


Figure 3 – Les deux consommateurs Kafka fonctionnent indépendamment : indexation et détection.

À retenir. Elasticsearch conserve les logs consultables ; Redis gère les compteurs et l'expiration de la blocklist ; MongoDB conserve les alertes et l'audit. L'interface affiche ensuite l'incident à l'analyste. En mode CORE, l'indexation ne passe pas par Kafka et la détection peut être effectuée périodiquement par le worker.

6. Architecture globale cible

La plateforme suit quatre étages : collecte, transport, traitement/stockage et interface. Le schéma représente le **mode STREAM** ; le **mode CORE** utilise un trajet simplifié précisé sous la figure.

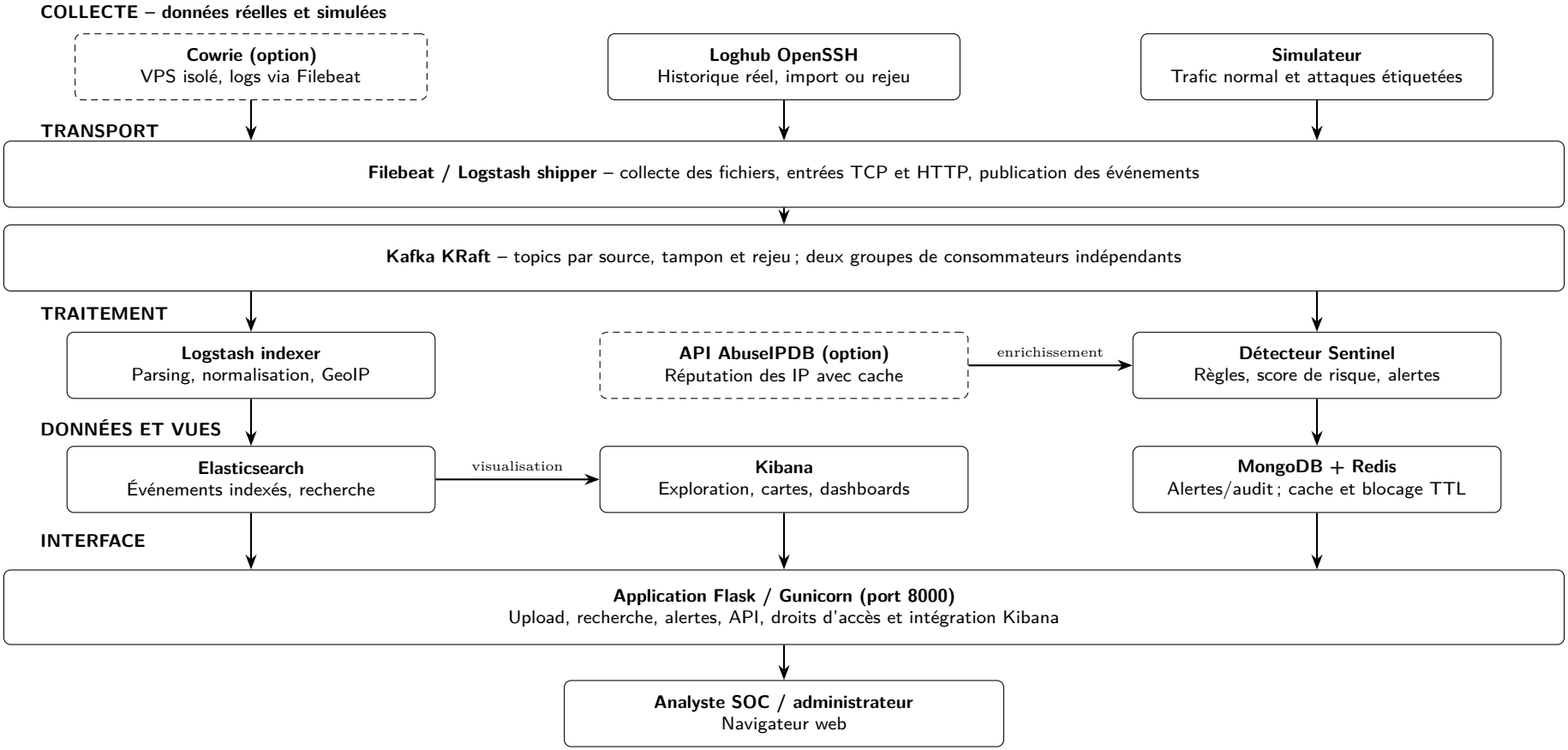


Figure 4 – Architecture logique du profil STREAM, à partir du schéma global de CampusGuard.

CORE (MVP). L'utilisateur importe un fichier dans Flask ; après validation, le dossier surveillé alimente Logstash, qui normalise puis indexe dans Elasticsearch. MongoDB gère les métadonnées, Redis le cache ; Kibana et Flask permettent la consultation. Aucun Kafka n'est nécessaire dans ce mode.

STREAM (extension). Filebeat, TCP ou HTTP alimentent le shipper et Kafka ; l'indexer et le détecteur consomment indépendamment les événements. Les alertes sont stockées dans MongoDB et propagées à l'interface via Redis.

7. Choix des composants

Composant	Rôle principal / justification
Flask / Gunicorn	API et pages web dans un même environnement Python, pratiques pour les modules de détection.
Logstash / Filebeat	Filebeat transporte les journaux ; Logstash assure parsing, normalisation et GeoIP avant indexation.
Elasticsearch	Indexation, recherche et agrégations sur les événements.
Kibana	Exploration et visualisation des indicateurs de sécurité.
MongoDB	Documents évolutifs : fichiers, règles, alertes et historique des enquêtes.
Redis	Données temporaires à accès rapide : sessions, cache, compteurs, diffusion live et blocklist TTL.
Celery	Exécution asynchrone des règles et notifications.
Kafka (STREAM)	Découple indexation et détection, conserve brièvement les événements et permet le rejeu après arrêt d'un service.
Docker Compose	Déploiement local reproductible, services et profils séparés.

7.1. Choix dárchitecture : CORE puis STREAM

CORE : priorité au parcours complet *fichier* → *validation Flask* → *Logstash* → *Elasticsearch* → *recherche/Kibana*. MongoDB conserve la traçabilité des imports ; Redis accélère les opérations temporaires.

STREAM : Kafka devient pertinent lorsque plusieurs sources et consommateurs fonctionnent simultanément : en cas d'arrêt de l'indexation, les événements conservés peuvent être rejoués. Le mode STREAM reste optionnel sur le PC de 12 Go et dispose d'un repli Redis pour préserver le MVP.

7.2. Critères de validation

La solution sera évaluée à partir de scénarios reproductibles :

— Un fichier CSV ou JSON correct est accepté ; au moins un événement apparaît dans Elasticsearch puis dans l'interface de recherche.
— Un fichier invalide est refusé avec un motif ; une alerte déclenchée possède un historique et un statut vérifiables.
— Les tests enregistrent le déclenchement ń 5 échecs / 5 min ż, la durée du blocage local, la mémoire et le comportement après redémarrage.